

Act as a Principal Business Intelligence Architect, Data Engineer, Analytics Engineer, Financial Systems Engineer, JavaScript Developer, SaaS Architect, Security Engineer, and Product Designer.

Your task is to build Layer 3 of an AI Business Decision Intelligence Platform as a standalone HTML application.

The layer is called:

BUSINESS INTELLIGENCE LAYER

This layer depends on:

1. Data Acquisition Layer
2. Business Operations Layer

Do not build the Digital Twin, Monte Carlo simulations, or AI Decision Intelligence yet.

Core Purpose

The application must transform validated operational records into trusted metrics, trends, anomalies, benchmarks, alerts, health scores, reports, and business explanations.

The layer must answer:

1. What happened?
2. Why did it happen?
3. Where is the business improving or deteriorating?
4. Which areas require attention?

It must not yet answer:

“What strategic decision should the owner take?”

Existing Architecture Compatibility

Use:

`window.INFINICUS.bus.register(route, handler, schema)`

`window.INFINICUS.bus.call(route, payload)`

`window.INFINICUS.bus.emit(event, data)`

`window.INFINICUS.bus.on(event, listener)`

All intelligence modules must communicate through the EngineBus.

Do not modify or recreate the existing simulation engine.

Data Dependencies

Read validated records from the Data Acquisition and Business Operations layers.

Expected sources include:

sales orders payments expenses inventory movements customers employees shifts suppliers purchase
orders tasks cash register records daily closings audit events

The Business Intelligence Layer must not create a second operational record system.

Required Modules

Build:

1. Intelligence workspace
2. Metric Registry
3. Aggregation Engine
4. Financial Intelligence
5. Sales Intelligence
6. Customer Intelligence
7. Inventory Intelligence
8. Operational Intelligence
9. Employee Intelligence
10. Supplier Intelligence
11. Branch Intelligence
12. Target management
13. Variance Engine
14. Trend Detection
15. Anomaly Detection
16. Benchmarking Engine
17. Business Health Score
18. Alert System
19. Explanation Engine
20. Executive Dashboard
21. Department dashboards
22. Reporting and export
23. Data-quality scoring
24. Digital Twin integration adapter

Metric Registry

Create one canonical registry for every KPI.

Each metric must define:

{ metricId, name, category, description, formula, unit, aggregation, frequency, direction, version }

Do not calculate the same KPI differently in multiple modules.

Initial Financial Metrics

Support:

grossRevenue netRevenue costOfGoodsSold grossProfit grossMargin operatingExpenses operatingProfit
netProfit netMargin cashInflow cashOutflow burnRate runway expenseRatio revenueGrowth profitGrowth

Initial Sales Metrics

Support:

totalOrders completedOrders cancelledOrders refundedOrders averageOrderValue salesByProduct
salesByCategory salesByBranch salesByEmployee salesByPaymentMethod discountRate refundRate
orderCompletionRate

Initial Customer Metrics

Support:

newCustomers returningCustomers activeCustomers repeatPurchaseRate purchaseFrequency
averageCustomerSpend retentionRate churnRate estimatedLifetimeValue customerAcquisitionCost
ltvCacRatio revenueConcentration

Initial Inventory Metrics

Support:

currentStock stockValue inventoryTurnover daysRemaining wasteRate damageRate shrinkage
stockoutFrequency overstock deadStock fastMovingProducts slowMovingProducts reorderUrgency

Required Intelligence Structures

Create canonical structures for:

metric values aggregations targets variances trends anomalies benchmarks health scores alerts
explanations reports intelligence snapshots

Business Health Score

Calculate:

financial health sales health customer health inventory health operational health workforce health supplier health

Use documented, versioned, industry-aware weights.

Do not hide formulas.

Trend Detection

Use transparent statistical techniques first:

moving averages percentage change rolling growth standard deviation linear regression slope consecutive increases or decreases

Return:

direction change percentage confidence period supporting values

Anomaly Detection

Detect:

unexpected revenue changes unusual expenses refund spikes cash variance abnormal inventory movement customer decline supplier price increases unusual employee activity

Every anomaly must include:

observed value expected range severity deviation score evidence status

Explanation Engine

Generate explanations containing:

observation supporting evidence likely contributing factors business impact confidence

Do not invent causes.

Clearly distinguish:

confirmed cause likely factor correlation insufficient evidence

EngineBus Namespace

Use:

intelligence.*

Register explicit routes for:

workspace metrics aggregations financial sales customers inventory operations employees suppliers
branches targets trends anomalies benchmarks health alerts explanations reports snapshots

Data Integrity

Implement:

business ownership validation branch isolation metric versioning period validation currency validation
duplicate aggregation prevention historical preservation traceability to source records alert deduplication
safe HTML rendering input sanitization error envelopes missing-data handling confidence scoring

User Interface

Use:

black background neon green accents white typography IBM Plex Sans IBM Plex Mono mobile-first
responsive design desktop left navigation accessible tables and forms clear metric definitions comparison
indicators visible data-confidence labels clear alert severity exportable reports

Desktop navigation:

Overview Financial Sales Customers Inventory Operations Employees Suppliers Branches Alerts Reports
Settings

Mobile navigation:

Overview Financial Sales Alerts More

Storage

Use:

localStorage for settings and display preferences IndexedDB for historical metric values, aggregations, alerts, and reports a clean adapter for future Firestore, Supabase, or another cloud database

Do not hardcode cloud credentials.

Build Method

Build in small executable blocks.

For each block:

1. State its purpose.
2. State dependencies.
3. Provide only the required code.
4. Explain exactly where to insert it.
5. Provide a manual test procedure.
6. Review accuracy, security, and data-integrity risks.
7. Stop after completing that block.

Do not regenerate the full project after every block.

Do not remove or rewrite previously working modules.

First Response

Do not begin coding immediately.

First provide:

1. Layer architecture.
2. Module dependency map.
3. Metric Registry design.
4. Canonical intelligence schemas.
5. EngineBus route map.
6. Event map.
7. Storage design.
8. Business Health Score method.
9. Trend and anomaly methods.
10. Exact development sequence.

After the architecture review, begin with Block 1 only.